

Vibe Coding Strategy for Using the LastOne Architecture Guide

May 13, 2026

Since you will be “vibe coding” (using an AI like Claude Code or Antigravity to write the syntax), you are in a great position. The LastOne Architecture Guide was written explicitly for your exact workflow: it focuses heavily on giving you the deep conceptual understanding required to direct an AI, rather than teaching you how to write the code yourself.

To use these two documents effectively without getting overwhelmed by unnecessary technical details, here is the step-by-step strategy you should follow:

1 Read the Architecture Guide Sequentially

Do not skip sections in the Architecture Guide, as each one assumes you have read the previous ones. Your goal here is to build a “mental model.” Since the AI will write the code, your primary job is to tell it where things should live and how they connect.

- Focus heavily on Section 1 (The Mental Model): You need to deeply understand the difference between Authentication (“who are you?”) and Authorization (“what are you allowed to do?”). You also must grasp the different types of “State” (server, UI, global) because telling the AI where to store data will prevent hard-to-fix bugs later.
- Don’t memorize syntax: When the guide covers React Native and TypeScript, remember that the AI will handle the heavy lifting. You do not need to learn TypeScript deeply; you just need to know that it helps the AI understand the shape of your data.
- Play the Flexbox game: The guide notes that layout bugs in React Native are almost always due to Flexbox. Spend 30 minutes on the recommended “Flexbox Froggy” game so you can accurately tell the AI how to fix UI layout issues.

2 Master “Priority 1” Concepts Before Building

Once you understand the overarching architecture, switch to the Technical Learning List. You must understand the “Priority 1” topics before you prompt the AI to write your first feature, or you will be blocked from day one.

- Understand React Query and Context: You need to conceptually know how your app fetches and caches database data (`useQuery`), how it writes data (`useMutation`), and how global data like user logins flows through the app.
- Learn enough to review, not write: For example, you don't need to know how to write complex SQL database migrations from scratch. You just need to understand basic SQL commands (`CREATE TABLE`, foreign keys, RLS policies) so you can review the AI's output and catch its mistakes.
- Set up your debugging workflow: Before you build anything, know how to check Metro bundler logs, use Expo DevTools, and read Supabase logs. When the AI writes broken code, these logs are what you will copy and paste back to the AI to help it fix the errors.

3 Scaffold and Follow the 12-Step Sequence

Once your Priority 1 learning is complete, you can begin the actual build.

- Copy the exact “Antigravity Scaffold Prompt” provided in Section 6.6 of the Architecture Guide to start your project.
- Follow the strictly ordered “First Build Steps” in Section 6.7. Do not jump ahead. Each step (like setting up the database, then building the auth flow, then building the home screen) is designed to validate your infrastructure before you build the next feature.

4 Learn “Just-in-Time” (Priority 2 & 3)

Do not try to learn everything else on the list right now. The Technical Learning List is designed as a “just-in-time” curriculum.

- Learn as you hit specific features: Only look up Keyboard Handling when you are building the Onboarding screens. Only learn about Reanimated 2 and Gesture Handling when you reach the Coping Library. Only research the complexities of Push Notifications and explicit App Permissions when you are ready to build the notifications step.
- Defer advanced items: Priority 3 items—like advanced Error Handling, Deep Linking, and Sentry Crash Reporting—are not immediate blockers. You can defer them until you are polishing the app (V2) or preparing for real user testing.

By reading the architectural concepts first, securing your Priority 1 operational knowledge, and deferring the rest until you specifically encounter those features in the 12-step build list, you will be able to efficiently guide your AI without getting bogged down in unnecessary tutorials.